

AgroLink — Revised Software Requirements Specification (SRS)

AI-Driven Agricultural Marketplace and Intelligence Portal

1. Executive Summary

AgroLink is a cloud-native agricultural platform connecting farmers and buyers through a digital marketplace while providing ML-driven insights for yield prediction, price estimation, and pest management. The system replaces Amazon Bedrock with free AI chatbot APIs (OpenAI/Anthropic/Gemini) and focuses on proven ML implementations with real agricultural datasets.

2. Core Features (Top 12)

Feature 1: User Management & Role-Based Access

- Multi-role registration (Farmer/Buyer/Admin)
- Authentication with JWT tokens
- Profile management with contact info and location

Feature 2: Farm Management

- Farmers create and manage multiple farms
- Track farm location, soil type, total acreage
- Upload farm images to S3

Feature 3: Crop Lifecycle Management

- Add crops with variety, planting date, expected harvest
- Track crop health status (Healthy/Stressed/Diseased/Harvested)
- Update crop stages through lifecycle

Feature 4: Inventory Tracking

- Monitor seeds, fertilizers, pesticides
- Automatic depletion on crop planting
- Low-stock alerts via CloudWatch

Feature 5: ML-Powered Yield Prediction

- Predict crop yield based on historical data
- Uses proven agricultural datasets
- Displays confidence intervals

Feature 6: ML-Powered Price Estimation

- Suggest optimal selling price

- Based on crop type, season, market trends
- Historical price visualization

Feature 7: Crop Marketplace

- Farmers list crops for sale with quantity and minimum price
- Browse available crops with filters (type, location, price)
- Crop availability status (Available/In Bidding/Sold)

Feature 8: Bidding System

- Buyers submit competitive bids
- Farmers view all bids with buyer ratings
- Accept/reject with one-click
- Auto-notification on bid status change

Feature 9: Order & Transaction Management

- Orders auto-generated on bid acceptance
- Payment tracking (Pending/Completed/Failed)
- Order history for both parties
- Transaction receipts stored in S3

Feature 10: Pest Analysis & Recommendations

- Upload pest images with crop context
- ML classification (Low/Medium/High severity)
- AI chatbot generates mitigation advice
- Historical pest report tracking

Feature 11: Financial Dashboard

- Auto-calculated revenue per crop sale
- Expense tracking (seeds, fertilizers, pesticides)
- Profit/loss statements per season
- Exportable financial reports (PDF to S3)

Feature 12: Analytics & Reporting

- Farmer performance metrics (yield vs predicted)
- Market trends visualization
- Seasonal comparison charts
- Admin oversight dashboard

3. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14 (React)	Server-side rendering, API routes

Styling	Tailwind CSS	Responsive UI components
Backend	Python FastAPI	REST API, ML inference
Database	PostgreSQL (RDS)	Relational data storage
File Storage	AWS S3	Images, reports, ML models
Compute	AWS EC2 (t2.micro)	Application hosting
Serverless	AWS Lambda	Event-driven automation
Monitoring	CloudWatch	Logs, metrics, alarms
AI Layer	OpenAI/Anthropic/Gemini API	Natural language explanations
ML Libraries	scikit-learn, pandas	Model training & inference

4. Machine Learning Implementation Plan

ML Task 1: Crop Yield Prediction

Dataset: [Crop Yield Prediction Dataset \(Kaggle\)](#)

- 28,000+ records
- Features: State, crop type, season, area, production
- Real agricultural data from India

Implementation:

1. Training Phase (One-time on local machine):

- Download dataset from Kaggle
- Clean and preprocess (handle missing values, encode categories)
- Train RandomForestRegressor with features: crop_type, soil_type, acreage, season, rainfall, temperature
- Evaluate with RMSE and R² metrics
- Save trained model as `yield_model.pkl` using joblib

2. Deployment:

- Upload `yield_model.pkl` to S3 bucket `/models/`
- EC2 backend loads model at startup
- Inference endpoint: `POST /api/predict-yield`

- Input: JSON with crop features
- Output: Predicted yield (tons), confidence interval

3. How It Works:

- User selects crop type, enters acreage, soil type
- Frontend sends request to EC2 backend
- Backend loads model from memory, runs prediction
- Returns result + stores in RDS `yield_predictions` table

ML Task 2: Market Price Estimation

Dataset: [Agricultural Commodity Prices \(Data.gov\)](#)

- Historical monthly grain prices (wheat, rice, maize, etc.)
- Time-series data suitable for trend analysis

Implementation:

1. Training Phase:

- Use ARIMA or Prophet for time-series forecasting
- Train separate models per crop type
- Include seasonality and trend components
- Save models as `price_model_{crop_type}.pkl`

2. Deployment:

- Upload models to S3 `/models/prices/`
- EC2 loads relevant model based on crop type
- Endpoint: `POST /api/predict-price`
- Input: `crop_type`, `current_month`, `quantity`
- Output: Suggested price range (min, max, recommended)

3. How It Works:

- Before listing crop, farmer requests price suggestion
- System analyzes historical trends for that crop/season
- Returns price range: "Based on market trends, suggested price: \$150-\$180/ton"
- Farmer uses this to set minimum bid price

ML Task 3: Pest Severity Classification

Dataset: [PlantVillage Dataset \(Kaggle\)](#) or simple rule-based classification

- For simplicity, use **rule-based classification** instead of deep learning
- Rules based on: `affected_area_percentage`, `crop_type`, `pest_type`

Implementation:

1. Rule-Based Approach (No training needed):

```
IF affected_area > 50% → High Severity
IF 20% < affected_area ≤ 50% → Medium Severity
```

IF affected_area $\leq$ 20% → Low Severity

Adjust thresholds based on crop sensitivity

2. Deployment:

- Logic implemented in Python backend
- No model file needed (just business rules)
- Endpoint: `POST /api/analyze-pest`
- Input: crop_type, pest_type, affected_area_percentage, image_url
- Output: severity_level, risk_score

3. How It Works:

- Farmer uploads pest image to S3
- Fills form: crop type, pest type, estimated affected area
- Backend applies rules, returns severity
- Image stored for future reference/training

AI Chatbot Integration (Replacing Bedrock)

Options: OpenAI API (GPT-4), Anthropic API (Claude), or Google Gemini API

Free Tier Availability:

- OpenAI: \$5 free credits for new accounts
- Anthropic: Limited free tier via API
- Gemini: Free tier available

Implementation:

1. Setup:

- Store API key in EC2 environment variables
- Create wrapper function in Python backend
- Rate limit: 10 requests/user/hour (CloudWatch tracking)

2. Use Cases:

- **Pest Mitigation Advice:** "Given [pest_type] on [crop_type] with [severity], suggest treatment"
- **Yield Explanation:** "Explain why yield is predicted at [X] tons for [crop] in [season]"
- **Price Justification:** "Why is the suggested price [X] for [crop] in [month]?"

3. API Call Flow:

User requests explanation
→ Backend formats prompt with context
→ Calls OpenAI/Claude/Gemini API

- Parses response
- Stores in RDS + returns to user

4. Cost Management:

- Cache common responses in RDS
- Use shorter models for simple queries
- Implement request quotas per user

5. Detailed Feature Implementation

Feature 1: User Management

Database Tables:

- `users` : id, name, email, password_hash, role, phone, location, created_at
- `user_sessions` : session_id, user_id, expires_at

Implementation:

1. Frontend (Next.js):

- Registration form with role selection
- Login page with JWT token storage
- Protected routes using middleware

2. Backend (FastAPI):

- `/api/register` - Hash passwords with bcrypt
- `/api/login` - Issue JWT tokens (24hr expiry)
- `/api/profile` - GET/PUT user details

3. AWS Services:

- **RDS**: Store user credentials
- **CloudWatch**: Log authentication attempts

Feature 2: Farm Management

Database Tables:

- `farms` : id, farmer_id (FK), name, location, soil_type, total_acreage, image_url, created_at

Implementation:

1. Frontend:

- Farm creation form (location dropdown, soil selector)
- Farm dashboard showing all farms
- Image upload component

2. Backend:

- `/api/farms` - POST: create farm, GET: list user's farms
- `/api/farms/{id}` - GET/PUT/DELETE specific farm

- Image upload: multipart form → S3 → return URL

3. AWS Services:

- **S3:** /farm-images/{farmer_id}/{farm_id}/
- **RDS:** Farm metadata
- **Lambda:** Trigger on farm creation to check subsidy eligibility

Feature 3: Crop Lifecycle Management

Database Tables:

- **crops** : id, farm_id (FK), crop_type, variety, acreage, planting_date, expected_harvest, actual_harvest, health_status, stage, created_at

Implementation:

1. Frontend:

- Crop creation form with date pickers
- Crop calendar view showing all crops by harvest date
- Health status indicator (color-coded badges)

2. Backend:

- /api/crops - CRUD operations
- /api/crops/{id}/update-stage - Update lifecycle stage
- Automatic health status based on pest reports

3. AWS Services:

- **RDS:** Crop records
- **Lambda:** Scheduled function (daily) to update crop stages based on planting_date

Feature 4: Inventory Management

Database Tables:

- **inventory** : id, farm_id (FK), item_type (seed/fertilizer/pesticide), item_name, quantity, unit, last_updated
- **inventory_transactions** : id, inventory_id (FK), transaction_type (add/use), quantity, timestamp

Implementation:

1. Frontend:

- Inventory table with add/use buttons
- Low stock warnings (< 10% of typical usage)

2. Backend:

- /api/inventory - GET inventory for farm
- /api/inventory/add - Add stock
- /api/inventory/use - Deduct stock (with transaction log)
- Auto-deduct when crop is planted (based on acreage)

3. AWS Services:

- **RDS:** Inventory records
- **CloudWatch:** Alarms for low stock (triggers SNS email)

Feature 5: ML-Powered Yield Prediction

Database Tables:

- `yield_predictions` : id, crop_id (FK), predicted_yield, confidence_score, model_version, created_at
- `yield_actuals` : id, crop_id (FK), actual_yield, recorded_at

Implementation:

1. Model Training:

- Use Crop Yield Dataset from Kaggle
- Features: crop_type, soil_type, acreage, season, avg_rainfall, avg_temp
- Train RandomForestRegressor in Jupyter notebook
- Achieve $R^2 > 0.75$ on test set
- Save model: `joblib.dump(model, 'yield_model.pkl')`

2. Deployment:

- Upload model to S3: `s3://agrolink-models/yield_model.pkl`
- EC2 backend loads model at startup using boto3
- Caches model in memory for fast inference

3. Frontend Flow:

- Farmer navigates to crop detail page
- Clicks "Predict Yield"
- Enters optional weather data (rainfall, temperature)
- Submits request

4. Backend Flow:

```
POST /api/predict-yield
→ Extract crop features from database
→ Prepare feature vector
→ model.predict(features)
→ Return: {predicted_yield: 12.5, confidence: 0.87, unit: "tons"}
→ Store in yield_predictions table
```

5. AI Explanation (Optional):

- After prediction, call OpenAI API
 - Prompt: "Explain in 2-3 sentences why this [crop_type] might yield [X] tons given [soil_type], [acreage], and [season]"
 - Display farmer-friendly explanation
-

Feature 6: ML-Powered Price Estimation

Database Tables:

- `price_predictions` : id, crop_id (FK), suggested_min, suggested_max, market_avg, created_at
- `historical_prices` : id, crop_type, price, date, source

Implementation:

1. Model Training:

- Dataset: Monthly Grain Prices ([Data.gov](#))
- Method: Use Facebook Prophet or ARIMA
- Features: crop_type, month, historical_avg
- Train per crop type (wheat, rice, maize, etc.)

2. Deployment:

- Upload models to S3: `/models/prices/wheat_price_model.pkl`
- Backend loads relevant model based on crop_type

3. Frontend Flow:

- Before listing crop, click "Get Price Suggestion"
- System shows: "Market analysis suggests \$150-\$180/ton"
- Farmer sets minimum bid price accordingly

4. Backend Flow:

```
POST /api/predict-price
→ Load price model for crop_type
→ Forecast next month's price
→ Calculate confidence interval
→ Return: {min: 150, max: 180, recommended: 165}
```

Feature 7: Pest Analysis & Recommendations

Database Tables:

- `pest_reports` : id, crop_id (FK), pest_type, affected_area_pct, severity, image_url, ai_advice, created_at

Implementation:

1. Rule-Based Severity:

```
def classify_severity(affected_area_pct, crop_type):
    if affected_area_pct > 50:
        return "High"
    elif affected_area_pct > 20:
        return "Medium"
    else:
        return "Low"
```

2. Frontend Flow:

- Upload pest image
- Select pest type from dropdown (aphids, locusts, blight, etc.)
- Estimate affected area (slider: 0-100%)
- Submit report

3. Backend Flow:

```
POST /api/pest-reports
→ Upload image to S3
→ Apply severity rules
→ Call AI chatbot API for mitigation advice
→ Store report in RDS
→ Return severity + recommendations
```

4. AI Chatbot Integration:

- Prompt: "A farmer reports [pest_type] on [crop_type] affecting [X]% of the field. Severity: [Y]. Provide 3-4 actionable mitigation steps."
- Response cached for similar pest/crop combinations
- Rate limit: 5 reports per farmer per day

5. AWS Lambda Trigger:

- **Trigger:** Image uploaded to S3 `/pest-images/`
- **Action:**
 - Extract metadata (crop_id, pest_type)
 - Call AI API for advice
 - Update pest_reports table with ai_advice
 - Send notification to farmer

Feature 8: Crop Marketplace

Database Tables:

- `marketplace_listings` : id, crop_id (FK), quantity_available, min_price, listing_status, listed_at

Implementation:

1. Frontend:

- Browse page with filters (crop type, location, price range)
- Search functionality
- Crop cards showing image, type, quantity, min price, farmer location

2. Backend:

- `/api/marketplace` - GET: list all available crops (listing_status = "active")

- `/api/marketplace/list` - POST: create listing (farmer only)
- `/api/marketplace/{id}` - GET: listing details

3. Business Rules:

- Only crops with `health_status = "Healthy"` or `"Harvested"` can be listed
- Quantity cannot exceed crop's total yield
- Listing auto-expires after 30 days

Feature 9: Bidding System

Database Tables:

- `bids` : `id`, `listing_id` (FK), `buyer_id` (FK), `bid_price`, `quantity`, `status` (pending/accepted/rejected), `created_at`
- `bid_notifications` : `id`, `bid_id` (FK), `user_id` (FK), `message`, `read_status`, `created_at`

Implementation:

1. Frontend (Buyer):

- Bid form on listing detail page
- Real-time bid status updates
- Bid history table

2. Frontend (Farmer):

- Incoming bids dashboard
- Accept/Reject buttons
- Buyer rating display

3. Backend:

- `/api/bids` - POST: submit bid (validates `price ≥ min_price`)
- `/api/bids/{id}/accept` - Creates order, updates listing
- `/api/bids/{id}/reject` - Updates bid status
- `/api/bids/my-bids` - GET bids by buyer
- `/api/bids/received` - GET bids for farmer's crops

4. Business Rules:

- Bid price must be $\geq$ `min_price`
- Quantity must be $\leq$ `quantity_available`
- Only one active bid per buyer per listing
- Accepting a bid auto-rejects other bids

5. AWS Lambda:

- **Trigger:** Bid status change in RDS
- **Action:**
 - Insert notification record

- Send email via SNS (optional)

Feature 10: Order Management

Database Tables:

- `orders` : id, bid_id (FK), farmer_id (FK), buyer_id (FK), crop_id (FK), quantity, total_price, order_status (pending/in_transit/delivered), payment_status (pending/completed/failed), created_at

Implementation:

1. Frontend:

- Order tracking page (farmer & buyer views)
- Status timeline (Pending → In Transit → Delivered)
- Payment confirmation button

2. Backend:

- `/api/orders` - GET: list orders for user
- `/api/orders/{id}/update-status` - PUT: update order_status
- `/api/orders/{id}/confirm-payment` - PUT: mark payment as completed

3. AWS Lambda - Financial Auto-Update:

- **Trigger:** order.payment_status changes to "completed"
- **Action:**
 - Calculate revenue: $\text{quantity} \times \text{total_price}$
 - Update `financials` table
 - Generate transaction receipt (PDF)
 - Upload receipt to S3: `/receipts/{order_id}.pdf`
 - Update crop quantity in marketplace

Feature 11: Financial Dashboard

Database Tables:

- `financials` : id, farmer_id (FK), total_revenue, total_expenses, net_profit, last_updated
- `transactions` : id, farmer_id (FK), type (revenue/expense), amount, category, description, date
- `expense_records` : id, farm_id (FK), category (seed/fertilizer/pesticide/labor), amount, date

Implementation:

1. Frontend:

- Dashboard with charts (revenue trend, expense breakdown)
- Monthly/seasonal filter
- Export to PDF button

2. Backend:

- `/api/financials/summary` - GET: aggregated stats
- `/api/financials/transactions` - GET: transaction history
- `/api/financials/export` - Generates PDF report

3. Auto-Calculation Logic:

- **Revenue:** Auto-added when order payment_status = "completed"
- **Expenses:** Manually added by farmer OR auto-deducted on inventory usage
- **Net Profit:** revenue - expenses (recalculated on every transaction)

4. AWS Lambda - Financial Report Generator:

- **Trigger:** CloudWatch scheduled event (monthly)
- **Action:**
 - Query transactions for each farmer
 - Generate PDF summary using ReportLab
 - Upload to S3: `/financial-reports/{farmer_id}/{month}.pdf`
 - Send notification

Feature 12: Analytics & Reporting

Implementation:

1. Farmer Analytics:

- Actual yield vs predicted yield comparison
- Crop success rate (healthy harvests / total crops)
- Revenue trends over time
- Most profitable crops

2. Market Analytics:

- Average bid prices by crop type
- Most traded crops
- Price fluctuation charts
- Seasonal demand patterns

3. Admin Dashboard:

- Total platform transactions
- Active users (farmers/buyers)
- Top-performing farmers
- System health metrics

4. Backend:

- `/api/analytics/farmer/{id}` - Farmer performance
- `/api/analytics/market` - Market trends
- `/api/analytics/admin` - Platform overview

5. AWS Services:

- **CloudWatch Dashboards:** Real-time system metrics
- **RDS Queries:** Aggregated analytics queries
- **S3:** Store generated chart images

6. AWS Lambda Functions (Event-Driven Automation)

Lambda 1: Pest Image Processing

Trigger: S3 PUT event on `/pest-images/` bucket

Function Logic:

```
def lambda_handler(event, context):
    # Extract S3 object key
    image_key = event['Records'][0]['s3']['object']['key']

    # Parse metadata (crop_id, pest_type from key)
    crop_id = extract_crop_id(image_key)

    # Get crop details from RDS
    crop = query_rds(f"SELECT * FROM crops WHERE id={crop_id}")

    # Call AI API for advice
    advice = call_openai_api(
        f"Suggest treatment for {pest_type} on {crop.crop_type}"
    )

    # Update pest_reports table
    update_rds(f"UPDATE pest_reports SET ai_advice='{advice}' WHERE image_id={image_key}")

    return {"status": "success"}
```

AWS Setup:

- Runtime: Python 3.11
- Timeout: 30 seconds
- Memory: 256 MB
- IAM Role: S3 read, RDS connect, OpenAI API secret access

Lambda 2: Order Completion Handler

Trigger: DynamoDB Stream / RDS Event or API Gateway trigger from backend

Function Logic:

```
def lambda_handler(event, context):
    order_id = event['order_id']

    # Get order details
    order = query_rds(f"SELECT * FROM orders WHERE id={order_id}")

    # Update financials
    revenue = order['total_price']
    update_rds(f"UPDATE financials SET total_revenue = total_revenue + {revenue}")

    # Generate receipt PDF
    receipt_pdf = generate_receipt(order)
    upload_to_s3(receipt_pdf, f"receipts/{order_id}.pdf")

    # Update marketplace Listing
    update_rds(f"UPDATE marketplace_listings SET quantity_available = quantity_available - {revenue}")

    return {"status": "processed"}
```

Lambda 3: Low Inventory Alert

Trigger: CloudWatch scheduled event (daily at 6 AM)

Function Logic:

```
def lambda_handler(event, context):
    # Query low stock items
    low_stock = query_rds("SELECT * FROM inventory WHERE quantity < 10")

    for item in low_stock:
        # Send SNS notification
        sns.publish(
            TopicArn='arn:aws:sns:region:account:low-inventory',
            Subject=f"Low Stock Alert: {item['item_name']}",
            Message=f"Only {item['quantity']} {item['unit']} remaining"
        )

    return {"alerts_sent": len(low_stock)}
```

Lambda 4: Crop Stage Auto-Update

Trigger: CloudWatch scheduled event (daily)

Function Logic:

```
def lambda_handler(event, context):
    today = datetime.now()

    # Update crops nearing harvest
    crops = query_rds("SELECT * FROM crops WHERE expected_harvest - CURRENT_DATE < 30")

    for crop in crops:
```

```

days_to_harvest = (crop['expected_harvest'] - today).days

if days_to_harvest <= 0:
    update_rds(f"UPDATE crops SET stage='Ready for Harvest' WHERE id={crop_id}")
elif days_to_harvest <= 7:
    update_rds(f"UPDATE crops SET stage='Maturing' WHERE id={crop_id}")
else:
    update_rds(f"UPDATE crops SET stage='Harvested' WHERE id={crop_id}")

return {"crops_updated": len(crops)}

```

7. AWS Architecture & Deployment

EC2 Setup (Application Server)

Instance Type: t2.micro (Free tier eligible)

Installation Steps:

1. Launch Ubuntu 24.04 EC2 instance
2. Install Python 3.11, Node.js 18, PostgreSQL client
3. Clone repository
4. Install dependencies:

```

pip install fastapi uvicorn boto3 psycpg2 scikit-learn joblib openai
npm install (in Next.js directory)

```

5. Configure environment variables:

```

DATABASE_URL=postgresql://user:pass@rds-endpoint/agrolink
AWS_REGION=us-east-1
S3_BUCKET=agrolink-storage
OPENAI_API_KEY=sk-...

```

6. Start services:

```

# Backend (FastAPI)
uvicorn main:app --host 0.0.0.0 --port 8000

# Frontend (Next.js)
npm run build && npm start

```

RDS Setup (PostgreSQL Database)

Instance Type: db.t3.micro (Free tier)

Configuration:

1. Create PostgreSQL 15 instance
2. Set master username/password
3. Enable public accessibility (for development)
4. Security Group: Allow port 5432 from EC2 security group
5. Create database: `agrolink`

6. Run schema migration scripts

S3 Bucket Structure

```
agrolink-storage/
├── farm-images/
│   ├── {farmer_id}/
│   │   └── {farm_id}/
├── pest-images/
│   ├── {crop_id}/
├── models/
│   ├── yield_model.pkl
│   ├── price_model_wheat.pkl
│   └── price_model_rice.pkl
├── receipts/
│   └── {order_id}.pdf
└── financial-reports/
    └── {farmer_id}/
```

Bucket Policies:

- Public read for farm-images (profile pictures)
- Private for all other folders
- EC2 IAM role: full access
- Lambda IAM role: read/write access

Lambda Deployment

For Each Lambda:

1. Create function in AWS Console
2. Runtime: Python 3.11
3. Upload code as .zip (include dependencies)
4. Set environment variables (DB credentials, API keys)
5. Configure trigger (S3 event, CloudWatch schedule, etc.)
6. Attach IAM role with necessary permissions

Example IAM Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::agrolink-storage/*"
    },
    {
      "Effect": "Allow",
      "Action": ["rds-data:ExecuteStatement"],
```

```
    "Resource": "*"
  }
]
}
```

CloudWatch Monitoring

Metrics to Track:

1. Application Metrics:

- API response times
- Request count per endpoint
- Error rates (4xx, 5xx)

2. ML Metrics:

- Prediction latency
- Model inference count
- Prediction accuracy (actual vs predicted)

3. Business Metrics:

- Daily active users
- Bids placed per day
- Orders completed per day

Alarms:

- CPU > 80% on EC2
- RDS connection count > 40
- Lambda error rate > 5%
- Low inventory (custom metric)

Logs:

- EC2 application logs → CloudWatch Logs
- Lambda execution logs (auto)
- RDS slow query logs

8. Database Schema (PostgreSQL)

```
-- Users
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) CHECK (role IN ('farmer', 'buyer', 'admin')),
  phone VARCHAR(20),
  location VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- Farms

```
CREATE TABLE farms (  
  id SERIAL PRIMARY KEY,  
  farmer_id INTEGER REFERENCES users(id) ON DELETE CASCADE,  
  name VARCHAR(100) NOT NULL,  
  location VARCHAR(100),  
  soil_type VARCHAR(50),  
  total_acreage DECIMAL(10,2),  
  image_url TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Crops

```
CREATE TABLE crops (  
  id SERIAL PRIMARY KEY,  
  farm_id INTEGER REFERENCES farms(id) ON DELETE CASCADE,  
  crop_type VARCHAR(50) NOT NULL,  
  variety VARCHAR(50),  
  acreage DECIMAL(10,2),  
  planting_date DATE,  
  expected_harvest DATE,  
  actual_harvest DATE,  
  health_status VARCHAR(20) DEFAULT 'Healthy',  
  stage VARCHAR(30) DEFAULT 'Planted',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Inventory

```
CREATE TABLE inventory (  
  id SERIAL PRIMARY KEY,  
  farm_id INTEGER REFERENCES farms(id) ON DELETE CASCADE,  
  item_type VARCHAR(20) CHECK (item_type IN ('seed', 'fertilizer', 'pesticide')),  
  item_name VARCHAR(100),  
  quantity DECIMAL(10,2),  
  unit VARCHAR(20),  
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Inventory Transactions

```
CREATE TABLE inventory_transactions (  
  id SERIAL PRIMARY KEY,  
  inventory_id INTEGER REFERENCES inventory(id),  
  transaction_type VARCHAR(10) CHECK (transaction_type IN ('add', 'use')),  
  quantity DECIMAL(10,2),  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Pest Reports

```

CREATE TABLE pest_reports (
    id SERIAL PRIMARY KEY,
    crop_id INTEGER REFERENCES crops(id),
    pest_type VARCHAR(50),
    affected_area_pct DECIMAL(5,2),
    severity VARCHAR(20),
    image_url TEXT,
    ai_advice TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Yield Predictions
CREATE TABLE yield_predictions (
    id SERIAL PRIMARY KEY,
    crop_id INTEGER REFERENCES crops(id),
    predicted_yield DECIMAL(10,2),
    confidence_score DECIMAL(3,2),
    model_version VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Yield Actuals (for model improvement)
CREATE TABLE yield_actuals (
    id SERIAL PRIMARY KEY,
    crop_id INTEGER REFERENCES crops(id),
    actual_yield DECIMAL(10,2),
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Price Predictions
CREATE TABLE price_predictions (
    id SERIAL PRIMARY KEY,
    crop_id INTEGER REFERENCES crops(id),
    suggested_min DECIMAL(10,2),
    suggested_max DECIMAL(10,2),
    market_avg DECIMAL(10,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Historical Prices
CREATE TABLE historical_prices (
    id SERIAL PRIMARY KEY,
    crop_type VARCHAR(50),
    price DECIMAL(10,2),
    date DATE,
    source VARCHAR(50)
);

-- Marketplace Listings

```

```

CREATE TABLE marketplace_listings (
    id SERIAL PRIMARY KEY,
    crop_id INTEGER REFERENCES crops(id),
    quantity_available DECIMAL(10,2),
    min_price DECIMAL(10,2),
    listing_status VARCHAR(20) DEFAULT 'active',
    listed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Bids
CREATE TABLE bids (
    id SERIAL PRIMARY KEY,
    listing_id INTEGER REFERENCES marketplace_listings(id),
    buyer_id INTEGER REFERENCES users(id),
    bid_price DECIMAL(10,2),
    quantity DECIMAL(10,2),
    status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Orders
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    bid_id INTEGER REFERENCES bids(id),
    farmer_id INTEGER REFERENCES users(id),
    buyer_id INTEGER REFERENCES users(id),
    crop_id INTEGER REFERENCES crops(id),
    quantity DECIMAL(10,2),
    total_price DECIMAL(10,2),
    order_status VARCHAR(20) DEFAULT 'pending',
    payment_status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Financials
CREATE TABLE financials (
    id SERIAL PRIMARY KEY,
    farmer_id INTEGER REFERENCES users(id),
    total_revenue DECIMAL(12,2) DEFAULT 0,
    total_expenses DECIMAL(12,2) DEFAULT 0,
    net_profit DECIMAL(12,2) DEFAULT 0,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Transactions
CREATE TABLE transactions (
    id SERIAL PRIMARY KEY,
    farmer_id INTEGER REFERENCES users(id),
    type VARCHAR(10) CHECK (type IN ('revenue', 'expense')),

```

```

    amount DECIMAL(10,2),
    category VARCHAR(50),
    description TEXT,
    date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

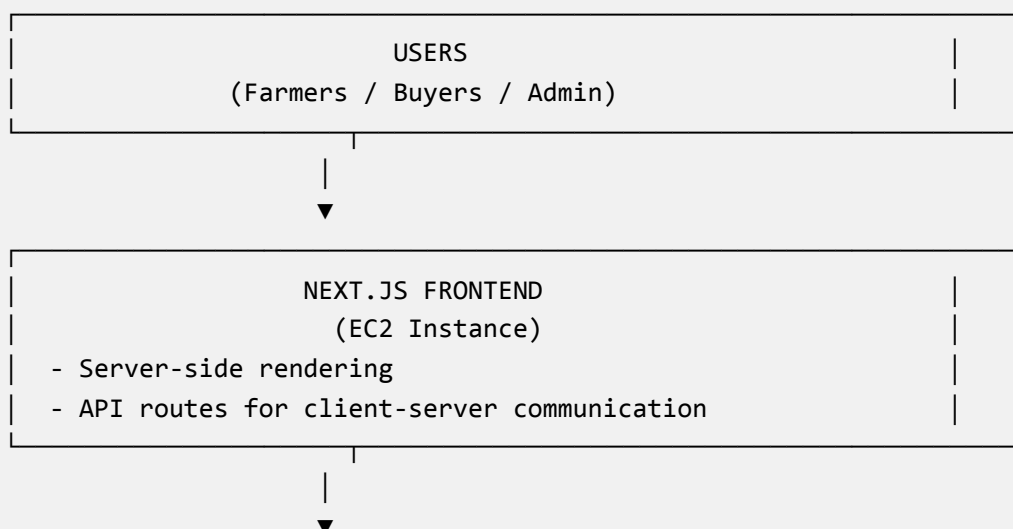
-- Expense Records
CREATE TABLE expense_records (
    id SERIAL PRIMARY KEY,
    farm_id INTEGER REFERENCES farms(id),
    category VARCHAR(30),
    amount DECIMAL(10,2),
    date DATE DEFAULT CURRENT_DATE
);

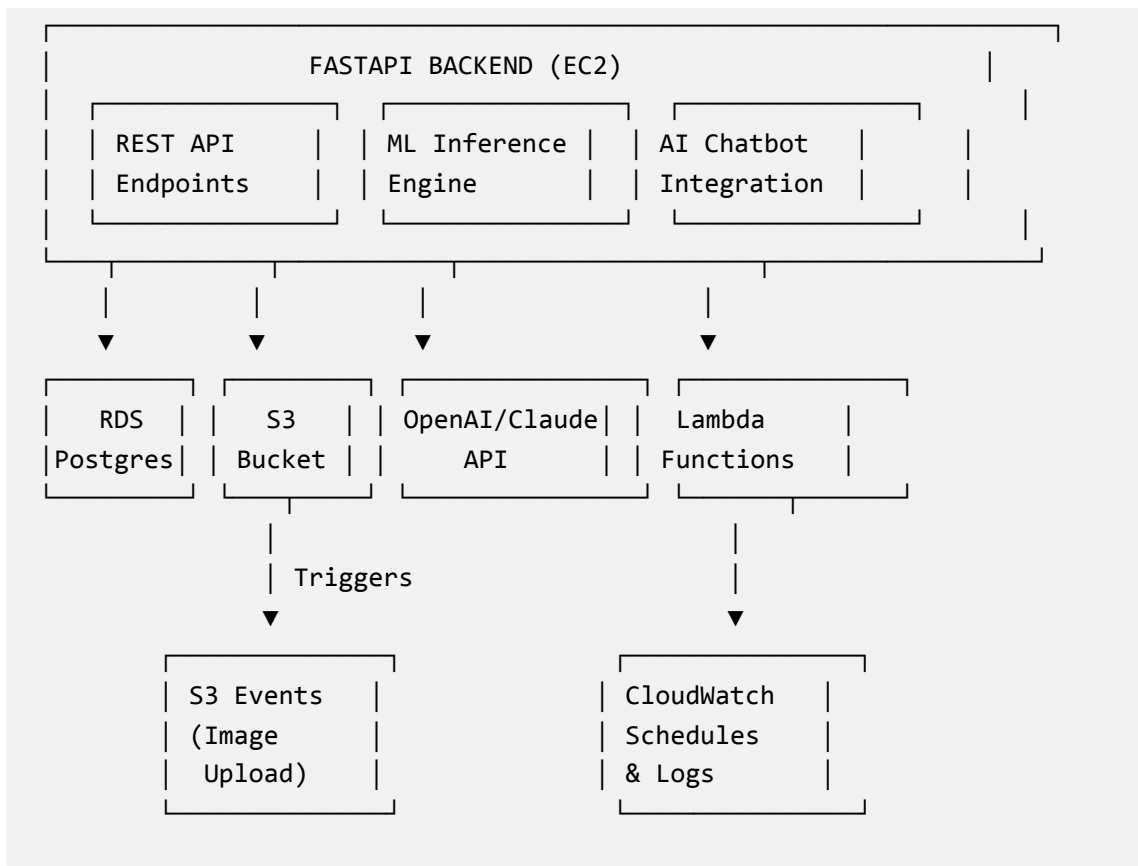
-- Bid Notifications
CREATE TABLE bid_notifications (
    id SERIAL PRIMARY KEY,
    bid_id INTEGER REFERENCES bids(id),
    user_id INTEGER REFERENCES users(id),
    message TEXT,
    read_status BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for Performance
CREATE INDEX idx_farms_farmer ON farms(farmer_id);
CREATE INDEX idx_crops_farm ON crops(farm_id);
CREATE INDEX idx_bids_listing ON bids(listing_id);
CREATE INDEX idx_orders_farmer ON orders(farmer_id);
CREATE INDEX idx_orders_buyer ON orders(buyer_id);
CREATE INDEX idx_marketplace_status ON marketplace_listings(listing_status);

```

9. System Architecture Diagram





10. Implementation Roadmap

Phase 1: Foundation (Week 1-2)

- Setup AWS account (Learner Lab)
- Deploy EC2, RDS, S3
- Implement user authentication
- Create database schema

Phase 2: Core Features (Week 3-4)

- Farm & crop management
- Inventory system
- Basic marketplace (list crops)

Phase 3: ML Integration (Week 5-6)

- Train yield prediction model
- Train price estimation model
- Deploy models to S3
- Implement inference endpoints

Phase 4: Marketplace & Bidding (Week 7-8)

- Complete bidding system
- Order management
- Payment tracking

Phase 5: Automation (Week 9)

- Deploy Lambda functions
- Configure S3 triggers
- Setup CloudWatch monitoring

Phase 6: AI & Analytics (Week 10)

- Integrate AI chatbot API
- Pest analysis with AI advice
- Financial dashboard
- Analytics reporting

11. Proven ML Datasets

ML Task	Dataset	Source	Size
Yield Prediction	Crop Yield Prediction	Kaggle Link	28K records
Price Estimation	USDA NASS Prices	Data.gov	10+ years
Pest Classification	PlantVillage (optional)	Kaggle	54K images

12. Cost Estimation (AWS Learner Lab)

Service	Usage	Monthly Cost
EC2 (t2.micro)	24/7	\$0 (free tier)
RDS (db.t3.micro)	750 hrs	\$0 (free tier)
S3	5 GB storage	\$0.12
Lambda	1M requests	\$0 (free tier)
CloudWatch	Basic logs	\$0 (free tier)
AI API	~1000 calls	\$0.50-\$2
Total		~\$0.50-\$2.50

Note: Learner Lab typically provides \$50-100 credits.

13. Key Differentiators

- ✓ **No Bedrock** — Uses free AI APIs (OpenAI/Claude/Gemini)
- ✓ **Proven Datasets** — Kaggle agriculture datasets with 28K+ records
- ✓ **Complete ML Pipeline** — Training → Deployment → Inference
- ✓ **AWS Best Practices** — EC2, RDS, S3, Lambda, CloudWatch

- ✓ **Event-Driven** — Lambda automation for business logic
 - ✓ **Production-Ready** — Scalable architecture, monitoring, logging
-

14. Next Steps

I can provide:

1. **ER Diagram** — Visual database relationships
2. **AWS Architecture Diagram** — Detailed component connections
3. **API Specification** — REST endpoint documentation
4. **ML Training Notebooks** — Step-by-step model training
5. **Lambda Function Code** — Production-ready Python scripts
6. **Deployment Guide** — Step-by-step AWS setup

Which would you like first?